Handoff — 2026-08-01

Written at the end of the night of 2026-07-31 → 2026-08-01. Read this first in a new session, then `docs/IMPLEMENTATION-PLAN-2026-08-01.md` for what to do next.

---

1. WHAT HAPPENED

Will played MAG in the **real Pokémon Showdown client** for the first time, six games, on a local server. This was the single most productive testing session the project has had: **eleven defects surfaced in about two hours**, and one human playing one evening found what 47 test files did not.

The reason it worked is worth keeping: the tests verify components in the configuration the tests choose. A person playing exercises the *shipped* configuration, end to end, and notices things no assertion was written for.

Record: MAG went 1–2 in the three games where the policy was actually driving. Games 1–3 do not count — the bot had its levers off and tested wiring only.

Nothing about MAG's playing strength was established. Six games is not evidence in either direction, and no work should be justified by that record.

---

2. STATE RIGHT NOW

Running. A Showdown server and the bot are both up. To restore after a reboot:

```
cd C:/Users/willj/Projects/Pokemon/pokemon-showdown && node pokemon-showdown start --no-security
```

```
cd C:/Users/willj/Projects/Pokemon/ABRA && SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown
```

Then open `http://localhost:8000`, log in as any name, and challenge **MAG** in `[Gen 9] Champions VGC 2026 Reg M-B`. The bot accepts only that format and declines anything else.

`SHOWDOWN_PATH` is **required** — without it `champions_sim.js` throws, because the Champions mod is not in the npm package and needs a built master checkout.

Killing an orphaned bot on Windows (`pkill` does not work here, and an orphan holds the name so the next login fails with `nametaken`):

```
Get-CimInstance Win32_Process -Filter "Name='node.exe'" | Where-Object { $_.CommandLine -like '*showdown_t
```

Bot configuration as shipped: `makeScoringPlayer({ greedy: true, joint: true })`. Voluntary `switching` is **deliberately off** — see §3.

Test suite, run at the end of this session (`node tests/run-all.js`):

```
46 passed, 1 failed, 1 skipped
```

- The failure is `tests/test-mag-page.js`, the **known Tier 2 red test** — `app/index.html`'s MAGIX scorer assigns 21 of 56 features, so 36 are silently 0, and 11 features disagree with the engine across 9 fixture cases. Pre-existing, unchanged by this session, and item 9 in §5.
- The skip is `engine/validate_selfplay.js` — no `data/games.selfplay.jsonl` on disk. **A skip is not a pass**, and the suite says so itself.
- `tests/test-wiring.js` **passed (13.7s) including the new assertions**, so the joint-path mega fix is verified by a test rather than by inspection.

---

3. EVERY CHANGE MADE THIS SESSION

Full evidence and measurements: `docs/FINDINGS-2026-08-01-live-play.md`.

file	change
`engine/magnemite.js`	`megaP` default 0 → **1**; the comment four lines above already said "Defaulted to 1"
`engine/magnemite.js`	**factory options now merge into constructor options.** `makeScoringPlayer(opts)` accepted `opts` and never used it — the class reads its own second constructor argument, so `makeScoringPlayer({greedy:true,switching:true})` configured **nothing**
`engine/magnemite.js`	both joint return paths now call `_withMega`. `_decidePair` returned bare, so `joint: true` **silently disabled mega evolution**
`engine/board.js`	`allyHit` now asks `getImmunity` before `getEffectiveness`. Type immunity is invisible to `getEffectiveness` (it returns 0 for immune AND for neutral), so a Flying partner beside Earthquake read as hit and `spreadFreeBesideAlly` **could never fire**
`engine/champions_sim.js`	Item Clause resample now drops `item` from `known` first. It was passing the colliding item back in, so all six redraws returned it and the "rare" no-item fallback fired **every time**
`engine/showdown_bot.js`	accepts `/acceptopenteamsheets`; passes `joint: true`; **removed** `switching: true`; bounded team redraw instead of rejecting the challenge
`tests/test-wiring.js`	mega, aiming and open sheets are now re-asserted **under** `--joint` — the joint path is a second code path and inherits no claim from the first

`switching` **was removed because it was already measured.** `mew.js:135`: *"Measured as a 10-point LOSS against a random opponent, so it is off until the switch policy is worth more than not switching."* The bot had asked for it since its first version; the options bug meant it never took effect until this session. Turning it on was an unmeasured change against a measured verdict. Post-KO replacement is a separate lever and is unaffected.

---

4. THE PATTERN — the most useful thing learned

Four defects had one shape: **knowledge that already existed in the repository, not applied at a new call site.**

defect	what the repo already said	what the new caller did
mega default	comment: *"Defaulted to 1"*	default was `0`
capability testing	`test-wiring.js` forces mega and open sheets ON	production passed neither
switching	`mew.js:135`: *"a 10-point LOSS … off until worth more"*	asked for it unconditionally
volatiles	`board.js:624`: *"Taunting into a Taunt looked identical to the first one"*	no feature ever consumed `hasVolatile`

Mega alone has now failed four times: never passed the option (`mew.js:446`, 199,524 games), left slot only (56% of sides), default 0, and bypassed by the joint path.

The root cause is not carelessness at any one site. It is that **defaults are set to the broken value and the working value is supplied by each caller individually**, so every new caller starts from broken. Where a working default exists, it should be the default.

---

5. TODO — in dependency order

Detail for every item: `docs/IMPLEMENTATION-PLAN-2026-08-01.md`.

Blocking

1. **REFIT.** `board.js` changed feature *semantics* under an unchanged feature *name*, so every shipped weight was fitted against the old meaning of `allyHit`. Refit both the 56-vector (`fit_policy.js`) and the 74-weight joint vector (`fit_joint.js`). `board.js` is required by **40 files**; the vector is read by **24**. **Blocks items 4 and 5.** Tests the hypothesis that the immunity bug is why `allyHit` sits at −0.0187 while `deadSide` is −2.879.

High leverage, cheap, not blocked

1. **A guard against feature MEANING changing.** `magnemite.js:297` compares feature names and `:299` compares vector length; both pass when a feature quietly starts meaning something else. Nothing in the project catches it. Proposal: a stored hash of each feature's values over a fixed fixture board, checked at weight load. **This is the only item that prevents a repeat rather than fixing an instance.**

1. `deadVolatile` **— and it CANNOT be a fitted feature.** MAG used Encore three times in one game, one of which failed. Every other "this move can't work now" case is covered (`deadStatus`, `deadSide`, `deadField`, `deadWeather`, `deadStall`, `deadNoLastMove`); volatiles are not. `board.js:620` **states the stored corpus cannot see volatiles**, so no weight can be learned from human replays. Implement as a **play-time candidate rule**, not a fitted weight — re-applying a live volatile does not *tend* to be worse, it definitionally fails. `hasVolatile()` already exists at `board.js:639`. No refit needed.

1. `bothSameSideCondition` **pair feature** *(needs 1)*. Two Light Screens on the same turn. `deadSide` cannot help — it asks whether the condition is *already up*, and on that turn it is up for neither slot. **Confirm first** that the stored corpus records both slots' choices on the same turn in a form `fit_joint.js` can use, or the weight will be estimated from nothing.

1. **A Choice-lock feature** *(needs 1)*. MAG switched out after Encore but not after Trick; nothing in the 56 features represents holding an unwanted Choice item. **Run the probe first** — the Encore switch may have been Showdown collapsing the legal move list rather than understanding, in which case a feature may not be the right fix. Also unresolved: whether mid-game item changes are in the stored corpus. If not, this is a play-time rule like item 3.

The bring — its own project

1. **The team-preview race.** Cheap, do it first: MAG sends `/choose team` **before** the opponent's `|showteam|` arrives, measured from the live protocol log. Hold the answer until the foe's sheet is seen, with a timeout that falls back to the current sampler and **counts** the fallback.

1. **The bring never looks at the opponent at all.** `chooseTeamPreview(team)` takes only its own team. It samples per-species marginals — *how often does anyone bring this* — never *should I bring it against THIS team*. Team selection is a large share of VGC skill and MAG has none of it. **Before building: re-read the recorded null result** that better beliefs did not improve the bring decision. If that experiment routed through this function, the null is close to guaranteed — an input a function does not read cannot improve its output — and it should be withdrawn rather than cited. Stated as a hypothesis; the experiment has not been read.

Team generation, no refit

1. **A support floor on sampled sets. 3,949 of 10,504 distinct sets were seen exactly once (37.6%).** Will drew a Gyarados running Smooth Rock / Sandstorm / Endeavor — one real human, once, out of 162 sheets. In `showdown_bot.js` **only**, sample within `species_sets.cover(sp, 0.80)`. **Not global**: DITTO's gauntlet must keep the full tail, because optimising against a field where everyone runs the modal set is optimising against a metagame that does not exist. `0.80` is a parameter and is stated as one — sweep it if it matters.

Carried over from before this session

1. Tier 2: `app/index.html`'s MAGIX scorer (21 of 56 features) and two hand-rolled damage calculators. Needs a dex shim plus build-time precomputation of handler results. A project, not a step; the 9 fixtures are the verification gate.
2. Apply the staged Protect fix to `spreadFreeBesideAlly`, then refit the pair block.
3. Wire `species_sets.sample()` into DITTO's gauntlet and rollouts.
4. Measure the self-play-retrained JOLTEON so evidence can move its tab.
5. `orb.html` refuses all 113 special moves; `index.html` / `orb.html` carry pasted dex copies.
6. G3 (generated ≠ hand-edited) and G4 (mutation registry) are not built.

---

6. ROADMAP POSITION

The end goal is unchanged: **a team that beats the meta**, with ALAKAZAM's Future Sight as the capstone. This session did not advance it. What it did was establish that **the substrate under it was not delivering the model to the board** — and that is a prerequisite, not a detour.

The honest sequencing now:

1. **Make the engine deliver the model.** Mostly done this session; item 2 keeps it done.
2. **Refit** (item 1), so the weights match what the features now mean.
3. **Then, and only then, measure MAG's strength.** No number produced before the refit describes the current engine.
4. **Then the bring** (items 6–7), because team selection is where "a team that beats the meta" is actually decided, and it is currently a matchup-blind sampler.
5. DITTO revival for team search, once the bring can express a matchup at all.

---

7. KNOWN BROKEN / GOTCHAS

- **The PDF toolchain is down.** `build/omnibus.py` cannot build PDFs on this machine: `OSError: cannot load library 'libgobject-2.0-0'` — WeasyPrint's GTK runtime is not on PATH, and there is no LibreOffice fallback installed. It worked on 2026-07-31, so something changed. Fixing it needs a system install, which is Will's call. **The review docs from this session are markdown only.** Note also that `omnibus.py` printed `omnibus PDF FAILED` and still **exited 0** — a silent failure of exactly the kind this project keeps hunting.
- **Never** `git add -A`**.** Will's ingest churns `data/archetypes.json`, `data/kad-replays.js`, `data/live.js` and `data/conformance.json`. They were dirty during this session and were **not** committed. Add files explicitly.
- **Never edit** `board.js`**,** `magnemite.js` **or** `engine-data.js` **while a fit or self-play run is in flight.** Checked before editing this session — nothing was running.
- **6 processes maximum.** 12 hard-crashed the machine; RAM is the real cap.

- **Will's Tower games are quarantined** — browser localStorage, stamped `tower-human`, never trained on.
- `web/` must stay byte-identical to `app/`.
- Check exit codes outside pipes.

---

8. THINGS I GOT WRONG THIS SESSION, SO THEY ARE NOT REPEATED

- **Claimed MAG had never mega evolved across 586,816 self-play games.** False. `mew.js:226` passes `mega` explicitly with a default of 1, and `test-wiring.js` measures megas on 85% of sides. The defect was confined to the Showdown bot. **The claim came from reading a default rather than running anything.**
- **Attributed the Earthquake-on-its-own-Blastoise to the `allyHit` weight of −0.019.** The larger cause was that the bot was *sampling* rather than taking its best move, because `greedy` never arrived.
- **Nearly wrote up a double Moonblast as overkill.** Checked: the first took Sableye to 50/100 and the second finished it. **Correct play.**
- **Nearly wrote up an immune move as a MAG error.** It was Will's own Prankster move failing into MAG's Dark-type Sableye.

The last two are why the rule is measure before claiming: two of the four "defects" I was about to record were the bot playing correctly.